Ders 8 Genişletilmiş Notu: Dinamik Tabloların Amortize Analizi ve Karmaşıklık Tekrarı

Ders 8 Genişletilmiş Notu: Dinamik Tabloların Amortize Analizi ve Karmaşıklık Tekrarı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dinamik Tablo Nedir?
- 2. Ekleme İşleminin Sözde Kodu
- 3. Toplam (Aggregate) Analiz ile Ekleme
- 4. Muhasebe (Accounting) Yöntemi ile Ekleme
- 5. Potansiyel (Potential) Yöntemi ile Ekleme
- 6. Silme İşleminin Eklenmesi ve Yanlış Bir Küçültme Stratejisi
- 7. Düzeltilmiş Strateji ve Yeni Potansiyel Fonksiyonu
- 8. Ekleme Durumlarının Analizi
- 9. Silme Durumlarının Analizi
- 10. Tekrar: Kaba Kuvvet ve İkili Arama
- 11. Tekrar: Ana Teorem
- 12. Tekrar: Hash Fonksiyonları ve Çakışma
- 13. Tekrar: Matematiksel Tümevarımla Yerine Koyma
- 14. Tekrar: Özyineleme Ağacı Yöntemi
- Örnek Sorular

1. Dinamik Tablo Nedir?

Hoca derse, geçen hafta anlatılan amortize analizin bir uygulamasını daha yapacağını, sonra da sınav öncesi tekrar için örnek çözeceğini söyleyerek başladı. Bu dersin uygulaması dinamik tablolardır (dynamic table): bir tabloya ekleme (insert) ve silme (delete) yapılırken amortize analizle karmaşıklığın ne olduğuna bakılacak ve öğrenilen üç amortize analiz yöntemi (toplam, muhasebe, potansiyel) aynı problem üzerinde uygulanacaktır. Anlatım Microsoft Whiteboard üzerinde yapıldı.

Bir tabloda herhangi bir anda değişen miktarda veri bulunabilir. Başta açılan yerden daha fazla veriye ihtiyaç duyulabilir, ya da daha azına. Boyutunu dinamik olarak büyütüp küçültmeye izin veren tablolara dinamik tablo denir. Tabloya rahat erişebilmek için tablonun bellek alanlarının art arda gelmesi gerekir. Bu yüzden tabloyu büyütmek, eski tablonun yanına birkaç yuva eklemek değil, daha büyük yeni bir tablo oluşturup eski elemanların hepsini ona taşımak demektir.

Dinamik tabloların bir kullanım alanı, önceki haftalarda görülen yeniden hash'lemedir (rehashing). Hash tablosunda doluluk oranını gösteren bir yük faktörü (load factor) vardı: $\alpha = n/m$, yani eleman sayısının yuva sayısına oranı. Bu oran aşılınca daha büyük bir tablo oluşturulur, eski elemanlar yeni hash değerleriyle yeni tabloya yerleştirilir ve yeni elemanlar da bu tabloya eklenmeye devam eder. Tersi de vardır: silmelerle doluluk belirli bir oranın altına inince tablo küçültülebilir.

Yeniden hash'lemede genel yaklaşım yeni alanı eskisinin iki katı açmaktır. Sistemde yalnız ekleme varsa, silme yoksa, tablonun doluluk oranı herhangi bir anda en az $1/2$'dir. Böylece boşa giden yer hiçbir zaman tablonun yarısını aşmaz.

2. Ekleme İşleminin Sözde Kodu

Analizler bir sözde kod üzerinden yapıldı. Tahtadaki tanımlar (ekrandan, 00:10:00):

- $\alpha = n/m$: doluluk oranı.
- `T`: tablo.
- `T.table`: tabloda saklanan bellek bloğunu gösteren işaretçi (pointer to block of storage).
- `T.num`: tablodaki eleman sayısı (# items in table).
- `T.size`: tablodaki yuva sayısı (# slots in table).

Başlangıçta tablo boştur: `T.num = T.size = 0`. Ekleme işlemi şöyledir (ekrandan, 00:13:20):

```
TABLE_INSERT(T, x)
  if T.size == 0
      allocate T.table with 1 slot
      T.size = 1
  if T.num == T.size
      allocate new_table with 2 * T.size slots
      insert all items in T.table into new_table
      free T.table
      T.table = new_table
      T.size = 2 * T.size
  insert x into T.table
  T.num = T.num + 1
```

Tablo boşsa önce tek yuvalık yer ayrılır ve `T.size` 1 yapılır; tablo bu alan üzerinden takip edilir. Eleman sayısı tablo boyutuna eşitse, yani tablo tamamen doluysa, genişletme (expansion) gerekir: iki kat yuvalı yeni tablo açılır, eski elemanların hepsi yeni tabloya eklenir, eski tablo serbest bırakılır (free), işaretçi yeni tabloya çevrilir ve `T.size` iki katına çıkarılır. Son olarak `x` tabloya eklenir ve eleman sayısı bir artırılır.

Hoca, çalışma süresinin sözde koddaki ekleme satırlarına göre, yani temel ekleme işlemlerinin sayısına göre değerlendirileceğini söyledi. İki ekleme satırı vardır: eski elemanları yeni tabloya taşıyan satır ve `x`'i ekleyen satır. Genişletmenin yapıldığı blok da analiz için önemlidir.

3. Toplam (Aggregate) Analiz ile Ekleme

c_i, i'inci işlemin gerçek maliyeti olsun. i'inci işlem ilk işlemse ya da tabloda yer varsa, yani genişletme yapılmıyorsa, tek bir ekleme yeterlidir: $c_i = 1$. i'inci işlem genişletmeye yol açıyorsa önceki $i - 1$ elemanın yeni tabloya taşınması ve i'inci elemanın eklenmesi gerekir: $c_i = (i - 1) + 1 = i$ (ekrandan, 00:18:50).

Genişletme her i değerinde olmaz. Tablo her dolduğunda iki katına çıkarıldığı için genişletme, $i - 1$ 2'nin kuvveti olduğunda olur:

$$c_i = \begin{cases} i & i-1 \text{ 2'nin kuvvetiyse} \\ 1 & \text{değilse} \end{cases}$$

Sistemde yalnız ekleme olduğuna göre n tane `TABLE_INSERT` işleminin toplam maliyeti, genişletme olmayan her işlemden gelen 1'ler ile genişletme maliyetlerinin toplamıdır. Tahtadaki ifade (ekrandan, 00:18:50):

$$\sum_{i=1}^{n} c_i \leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j$$

Hoca, ilk terimin her işlemin bir birimlik ekleme payını, ikinci terimin taşıma maliyetlerini verdiğini, ifadenin gerçek toplamdan biraz büyük olduğunu, bu yüzden eşitlik değil üst sınır yazıldığını açıkladı. Geometrik seri $\sum_{j=0}^{k} 2^j = 2^{k+1} - 1$ olduğundan ve $2^{\lfloor \lg n \rfloor} \leq n$ olduğundan ikinci terim $2n$'den küçüktür (ara adım eklendi). Sonuç:

$$\sum_{i=1}^{n} c_i < n + 2n = 3n$$

Yani n eklemeden oluşan bir dizinin toplam maliyeti $3n$'den küçüktür; işlem başına amortize maliyet $O(1)$'dir. Kullanılan yöntem toplam (aggregate) analizdir.

Örnek olarak ilk dokuz işlemin gerçek maliyetleri 1, 2, 3, 1, 5, 1, 1, 1, 9'dur: genişletmeler $i = 2, 3, 5, 9$ adımlarında olur. $n = 16$ için toplam $16 + (1 + 2 + 4 + 8) = 31 < 48$'dir. Sınır $n = 1, 16, 17, 100, 1024, 1025$ için python ile denendi ve hep sağlandı ($n = 1025$ için toplam 3072, sınır 3075).

4. Muhasebe (Accounting) Yöntemi ile Ekleme

Hoca muhasebe yöntemini hatırlattı: yığın örneğinde restoranlardaki üst üste konan tabaklardan söz edilmiş, PUSH işlemine fazladan kredi verilmiş, çekme işlemlerinin maliyeti bu krediden karşılanmıştı. Burada da benzer bir şey yapılır: her eklemeye 3 kredi biçilir ve bunun gereksinimi karşılayıp karşılamadığına bir şekil üzerinden bakılır.

Şekil (ekrandan, 00:26:00): Tabloda x_1, x_2, x_3 varken ($i = 3$) dördüncü işlem gelir, x_4 eklenir ve tablo dolar. Beşinci işlem geldiğinde tablo dolu olduğu için genişletme yapılır: $x_1, \ldots, x_4$ yeni tabloya taşınır ve x_5 eklenir. Hoca anlatıma tam bu noktadan başladı: genişletmeden hemen sonra toplam kredi 0'dır.

x_5'e 3 kredi biçilir:

- Biri x_5'in kendisini eklemenin gerçek maliyetidir.
- Biri x_5'in üstünde kalır; bir sonraki genişletmede x_5'in taşınmasını karşılar.
- Biri eski yarıdaki bir elemanın, örneğin x_1'in üstüne yazılır; onun bir sonraki taşınmasını karşılar.

x_6, x_7, x_8 de birer 3 kredi ile eklenir. Tablo yeniden dolduğunda x_1 ile x_8 arasındaki her elemanın üstünde birer kredi vardır. Dokuzuncu işlemde x_9 gelince yeniden genişletme gerekir ve bu birer krediler elemanları yeni tabloya taşımak için harcanır. Bu noktada toplam kredi yine 0 olur. Sonra x_9 için yine 3 kredi verilir: biri kendisi için, biri x_1'in, biri de x_9'un bir sonraki genişletmedeki taşınması için.

Bu sistemde her işlemin amortize maliyeti sabit bir sayıdır. n işlem varsa toplam $O(n)$ olur.

5. Potansiyel (Potential) Yöntemi ile Ekleme

5.1 Potansiyel Fonksiyonunun Seçilmesi

Potansiyel yöntemi için bir potansiyel fonksiyonu gerekir. Hoca aradığı fonksiyonun özelliklerini şöyle koydu: genişletmeden hemen sonra değeri sıfırlanmalı, toplam analizde olduğu gibi, ve tablo doldukça yavaş yavaş değer kazanmalı. Önerilen fonksiyon (ekrandan, 00:30:00):

$$\Phi(T) = 2 \cdot T.num - T.size$$

Fonksiyon şöyle sınandı:

- Genişletmeden hemen sonra $T.num = T.size/2$ olur ve $\Phi(T) = 0$'dır. Beklenti karşılanır.
- Genişletmeden hemen önce $T.num = T.size$ olur ve $\Phi(T) = T.num$'dır. Eleman sayısı arttıkça potansiyel artar; genişletme anında taşınacak eleman sayısı kadar potansiyel birikmiştir.

- Tablo her zaman en az yarı yarıya dolu olduğu için ($T.num \geq T.size/2$) fonksiyon hiç negatif olmaz. Potansiyel yönteminin koşulu da potansiyelin negatife inmemesiydi.

5.2 Gösterim

İki ayrı durum arasındaki potansiyel farkı üzerinden hesap yapılacağı için şu değişkenler tanımlandı (ekrandan, 00:35:10):

- num_i: i'inci işlemden sonra tabloda saklanan eleman sayısı.
- $size_i$: i'inci işlemden sonra tablonun toplam boyutu.
- Φ_i: i'inci işlemden sonraki potansiyel.

Amortize maliyet, önceki dersteki formülle hesaplanır: $\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$. i'inci işlemin genişletmeye yol açtığı ve açmadığı iki durum vardır; hoca, en kötüsünü seçebilmek için ikisinin de incelenmesi gerektiğini söyledi.

5.3 Durum 1: Genişletme Yok

Genişletme olmadığı için tablo boyutu değişmez: $size_i = size_{i-1}$. Gerçek maliyet $c_i = 1$'dir. Ekleme yapıldığı için $num_{i-1} = num_i - 1$'dir. Formül:

$$\begin{aligned}\hat{c}_i &= 1 + (2num_i - size_i) - (2num_{i-1} - size_{i-1}) \\ &= 1 + 2num_i - size_i - 2num_i + 2 + size_i = 3\end{aligned}$$

Hoca hesabı yaparken önce bir işaret hatası yaptı, sonra fark edip düzeltti ve sonucun 3 olduğunu yazdı (ekrandan, 00:35:10). Yani ekleme işleminin amortize maliyeti sabit bir sayıdır.

5.4 Durum 2: Genişletme Var

Genişletme olunca yeni boyut eskisinin iki katıdır: $size_i = 2 \cdot size_{i-1}$. Genişletmeden önce tablo tamamen dolu olduğu için $size_{i-1} = num_{i-1} = num_i - 1$'dir. Buradan $size_i = 2(num_i - 1)$ çıkar (ekrandan, 00:40:00).

Gerçek maliyet, bölüm 3'teki $c_i = i$'dir; bu da i'inci işlemden sonraki eleman sayısı olan num_i'ye eşittir ($(i-1)$ taşıma ve 1 ekleme). Formülde $size_i$ yerine $2(num_i - 1)$, $size_{i-1}$ ve num_{i-1} yerine $num_i - 1$ konur:

$$\begin{aligned}\hat{c}_i &= num_i + (2num_i - 2(num_i - 1)) - (2(num_i - 1) - (num_i - 1)) \\ &= num_i + 2 - (num_i - 1) = 3\end{aligned}$$

Son sadeleştirmeyi hoca tahtada yazmadı, sonucun yine 3 olduğunu söyledi (ara adım eklendi). Sonuç: genişletme olsa da olmasa da işlemin amortize maliyeti sabit bir sayıyla sınırlıdır. İşlem başına $O(1)$'dir.

> **Ders sonrası not:** Hoca burada "N işlem için de O1'e tekabül edecek" dedi (00:40:10); doğrusu işlem başına $O(1)$, n işlem için toplam $O(n)$'dir.

6. Silme İşleminin Eklenmesi ve Yanlış Bir Küçültme Stratejisi

Şimdi sistemde hem ekleme hem silme vardır. Hoca önceki derste söylenenleri hatırlattı: n işlemin içinde farklı türde işlemler varsa en kötü durumu veren diziyi analiz etmek gerekir. Ekleme ve silmelerin hangi sırada, hangisinden kaç tane geleceği düşünülmelidir.

Silme için eleman tablodan basitçe kaldırılır, ama yer tasarrufu da gereklidir. Doluluk oranı çok düşerse daha küçük bir tablo açıp elemanları ona taşımak genel yaklaşımdır. Maliyet yine ekleme ve silme işlemlerinin sayısıyla hesaplanır.

Akla gelen ilk strateji şudur: tablo dolunca boyut iki katına çıkarılır (önceki gibi), bir silme tabloyu yarıdan az dolu hâle getirirse, yani doluluk $1/2$'nin altına inerse, boyut yarıya indirilir. Bu stratejide toplam maliyet nereye gider?

n işlem olsun ve n 2'nin kuvveti olsun (ekrandan, 00:50:00). İlk $n/2$ işlem ekleme olsun. Bunun maliyeti önceki bölümlerde hesaplandı: $\Theta(n)$. Sonunda $T.num = T.size = n/2$ olur, tablo tamamen doludur. Sonraki $n/2$ işlem şu düzende gelsin:

ekle, sil, sil, ekle, ekle, sil, sil, ekle, ekle, ...

Hoca bu diziyi şekil çizerek izledi:

- İlk ekleme geldiğinde tablo dolu olduğu için genişletme yapılır. Yeni tablonun $n/2$'lik yarısı dolu, diğer yarısının ilk yuvası doludur.
- Birinci silme yeni eklenen elemanı siler; tablonun ilk yarısı tam dolu kalır.
- İkinci silme ilk yarıdan da bir eleman siler; doluluk $1/2$'nin altına iner ve tablo küçültülür (contraction).
- Sonraki iki eklemenin birincisi küçük tablonun son yuvasını doldurur, ikincisi yine genişletmeye yol açar.
- Sonraki iki silme yine küçültmeye yol açar, ve bu böyle sürer.

Sürekli büyütme ve küçültme yapılır. Her genişletme ve her küçültme n eleman taşıdığı için $\Theta(n)$ maliyetlidir ve bunlardan $\Theta(n)$ tane vardır. Toplam maliyet $\Theta(n^2)$'ye çıkar.

Bu stratejinin kötü yanı şudur: genişletmeden sonra küçültmenin maliyetini karşılayacak kadar silme yapılmıyor, küçültmeden sonra da genişletmenin maliyetini karşılayacak kadar ekleme yapılmıyor. Toplam analizde maliyet bir sonraki genişletmeye kadar yapılan eklemelerden karşılanıyordu; burada araya giren işlem sayısı buna yetmiyor.

Bu dizi $n = 64, 256, 1024$ için python ile denendi; toplam maliyet yaklaşık $n^2/8$ olarak büyüdü (sırasıyla 599, 8543, 132479), yani gerçekten $\Theta(n^2)$'dir.

7. Düzeltilmiş Strateji ve Yeni Potansiyel Fonksiyonu

Çözüm, genişletme için yine tam doluluğu kullanmak, ama küçültmede doluluğun $1/2$'nin daha da altına inmesine izin vermektir: ekleme ile tablo dolunca boyut iki katına çıkar, tablo $1/4$'ten daha az dolu hâle gelince boyut yarıya indirilir. Bu sistemde n işlemlik bir ekleme-silme dizisinin maliyeti yine potansiyel yöntemiyle analiz edildi.

Potansiyel fonksiyonu iki parçalıdır (ekrandan, 00:53:40):

$$\Phi(T) = \begin{cases} 2 \cdot T.num - T.size & \alpha(T) \geq 1/2 \\ T.size/2 - T.num & \alpha(T) < 1/2 \end{cases}$$

Birinci parça, yalnız ekleme için kurulan fonksiyonun aynısıdır. Genişletmeden hemen sonra $T.num = T.size/2$ olduğu için değeri 0'dır. Doluluk 1 olunca $T.size = T.num$ ve $\Phi = T.num$ olur; bir sonraki genişletmenin maliyetini karşılar.

İkinci parça yeni eklendi ve doluluğun $1/2$'nin altına düştüğü durumu kapsar. Örneğin $\alpha = 1/4$ iken $T.size = 4 \cdot T.num$ olur ve $\Phi = 2 \cdot T.num - T.num = T.num$ çıkar. Yani küçültme anında, taşınacak eleman sayısı kadar potansiyel birikmiştir. Hoca bu fonksiyonun ihtiyacı karşıladığını söyledi.

Gösterim önceki gibidir: c_i gerçek maliyet, $\hat{c}_i$ amortize maliyet, num_i eleman sayısı, $size_i$ tablo boyutu, Φ_i potansiyel. Yeni olarak $\alpha_i = num_i/size_i$, yani i'inci işlemden sonraki doluluk oranı da kullanılır. Başlangıçta $num_0 = 0$, $size_0 = 0$, $\Phi_0 = 0$'dır; doluluk oranı ise 0/0 bölmesi düşünülmeden 1 olarak başlatılır.

Önce i'inci işlemin ekleme olduğu, sonra silme olduğu durumlar incelendi. Hoca bunu yığın örneğindeki PUSH, POP ve MULTIPOP'a tek tek bakılmasına benzetti. Fonksiyon doluluk oranına göre farklı değerler aldığı için her işlem türünde doluluk durumlarının da tek tek ele alınması gerekir.

> **Hocanın vurgusu:** Formül çıkarıldıktan sonra bütün durumlar tek tek denenmelidir. Durumları denerken kritik olan, o durumun bir genişletmeye ya da küçültmeye yol açıp açmadığını belirlemek ve eşitlikleri buna göre yazmaktır (01:05:27).

8. Ekleme Durumlarının Analizi

8.1 $\alpha_{i-1} \geq 1/2$

Yalnız ekleme varken doluluk hep en az $1/2$ idi; bu durum bölüm 5'te zaten incelendi. Genişletme olsa da olmasa da $\hat{c}_i$ en fazla 3'tür. Hoca bunu söylemeyi unuttuğunu belirterek araya ekledi.

8.2 $\alpha_{i-1} < 1/2$ ve $\alpha_i < 1/2$

Yalnız ekleme varken bu durum hiç gerçekleşmiyordu; silme işin içine girince doluluk $1/2$'nin altına inebilir. Doluluk $1/2$'nin altındayken ekleme genişletmeye yol açmaz (genişletme ancak doluluk 1 olunca olur), bu yüzden $c_i = 1$, $size_i = size_{i-1}$ ve $num_{i-1} = num_i - 1$'dir. İki durumda da fonksiyonun ikinci parçası kullanılır (ekrandan, 01:00:00):

$$\begin{aligned}\hat{c}_i &= 1 + \left(\frac{size_i}{2} - num_i\right) - \left(\frac{size_{i-1}}{2} - num_{i-1}\right) \\ &= 1 + \frac{size_i}{2} - num_i - \frac{size_i}{2} + num_i - 1 = 0\end{aligned}$$

Terimler birbirini götürür ve amortize maliyet 0 çıkar.

8.3 $\alpha_{i-1} < 1/2$ ve $\alpha_i \geq 1/2$

Önceden tablo yarıdan az doluymuş, ekleme sonrası yarı yarıya dolu hâle gelmiş. Hoca önce genişletme olup olmadığını sordu ve şekille gösterdi: tablonun yarısına kadar dolu bir kısma bir eleman eklenmiş; genişletme yoktur. Bu yüzden $c_i = 1$ ve $size_i = size_{i-1}$'dir. Sonraki durumda fonksiyonun birinci parçası, önceki durumda ikinci parçası kullanılır. $num_i = num_{i-1} + 1$ yazılır (ekrandan, 01:08:30):

$$\begin{aligned}\hat{c}_i &= 1 + (2num_i - size_i) - \left(\frac{size_{i-1}}{2} - num_{i-1}\right) \\ &= 1 + (2(num_{i-1} + 1) - size_{i-1}) - \left(\frac{size_{i-1}}{2} - num_{i-1}\right) \\ &= 1 + 2num_{i-1} + 2 - size_{i-1} - \frac{size_{i-1}}{2} + num_{i-1} \\ &= 3num_{i-1} - \frac{3size_{i-1}}{2} + 3\end{aligned}$$

$\alpha_{i-1} = num_{i-1}/size_{i-1}$ olduğundan num_{i-1} yerine $\alpha_{i-1} \cdot size_{i-1}$ yazılır:

$$\hat{c}_i = 3\alpha_{i-1}size_{i-1} - \frac{3}{2}size_{i-1} + 3$$

Bu durumda $\alpha_{i-1} < 1/2$ olduğu için ilk iki terimin toplamı negatiftir. Dolayısıyla $\hat{c}_i < 3$'tür. Amortize maliyet yine en fazla 3'tür. İki durumda da eklemenin sabit bir sayıyla sınırlı olduğu görüldü.

9. Silme Durumlarının Analizi

Silmede eleman sayısı bir azalır: $num_i = num_{i-1} - 1$ (ekrandan, 01:10:00). Önce önceki doluluğun $1/2$'den küçük olduğu durum incelendi. Burada bir silme küçültmeye yol açabilir de, açmayabilir de; iki alt durum vardır.

9.1 $\alpha_{i-1} < 1/2$, Küçültme Yok

Küçültme yoksa tablo boyutu değişmez: $size_i = size_{i-1}$. Gerçek maliyet $c_i = 1$'dir ve $num_{i-1} = num_i + 1$'dir (ekrandan, 01:12:00):

$$\hat{c}_i = 1 + \left(\frac{size_i}{2} - num_i\right) - \left(\frac{size_{i-1}}{2} - num_{i-1}\right)$$
$$= 1 + \left(\frac{size_i}{2} - num_i\right) - \left(\frac{size_i}{2} - (num_i + 1)\right) = 2$$

Yine sabit bir sayı çıktı.

> **Ders sonrası not:** Tahtada ikinci satırın son parantezi $(size_i/2 - num_i + 1)$ diye yazıldı (ekrandan, 01:12:00). Doğrusu $(size_i/2 - (num_i + 1))$'dir; tahtadaki sonuç (2) bu doğru yazımla çıkar.

9.2 $\alpha_{i-1} < 1/2$, Küçültme Var

Küçültme olursa gerçek maliyet, o andaki elemanların yeni yere taşınması ve bir elemanın silinmesidir: $c_i = num_i + 1$. Formül:

$$\hat{c}_i = (num_i + 1) + \left(\frac{size_i}{2} - num_i\right) - \left(\frac{size_{i-1}}{2} - num_{i-1}\right)$$

Burada $size_i$, $size_{i-1}$, num_i ve num_{i-1} arasındaki ilişkiyi kurmak bir tık daha karmaşıktır. Hoca bunu bir şekille gösterdi (ekrandan, 01:20:00): Tabloda 8 yuva olsun ve $(i-1)$'inci işlemden sonra 2'si dolu olsun: $size_{i-1} = 8$, $num_{i-1} = 2$. i'inci işlem bir silmedir; bir eleman silinince $num_i = 1$ olur, doluluk $1/4$'ün altına düşer ve küçültme yapılır: $size_i = 4$.

Bu sayılar arasındaki ilişki şöyledir:

$$\frac{size_i}{2} = \frac{size_{i-1}}{4} = num_{i-1} = num_i + 1$$

Örnekte $4/2 = 8/4 = 2 = 1 + 1$'dir. Yani önceki doluluk tam $1/4$ sınırındadır ve bir eleman çıkınca sınırın altına düşülür. Bu eşitlikler yerine konunca:

$$\hat{c}_i = (num_i + 1) + ((num_i + 1) - num_i) - ((2num_i + 2) - (num_i + 1))$$
$$= num_i + 1 + 1 - num_i - 1 = 1$$

Küçültmeli silmenin amortize maliyeti de sabittir.

9.3 Kalan Durum

Hoca, $\alpha_{i-1} \geq 1/2$ iken silme durumunu derste çözmedi ve öğrencilere bıraktı. Bu durum yapılınca yine sabit bir sayı çıkacağını, böylece bütün ekleme ve silme seçenekleri için işlem başına $O(1)$, n işlem için $O(n)$ bulunacağını söyledi. Çözüm "Örnek Sorular" bölümündedir. Hoca, işlemleri takip etmekte zorlanılabileceğini, bu bölümün kendi başına tekrar edilmesini önerdi.

10. Tekrar: Kaba Kuvvet ve İkili Arama

Aradan sonra hoca sınav öncesi tekrar için sorular çözdü.

10.1 Doğru Üzerinde En Yakın Çift

Kaba kuvvet (brute force) algoritmalarında bir problemi çözmek için bütün seçenekler denenir ve biri seçilir. Hesap maliyeti yüksektir ama gerçeklemesi kolaydır.

Soru (ekrandan, 01:40:00): Gerçek doğru üzerinde $x_1, x_2, \ldots, x_n$ noktaları var. Birbirine en yakın iki nokta (closest pair) bulunacak. Kaba kuvvetle çözülürse karmaşıklık ne olur, daha iyi bir algoritma nasıl tasarlanır?

Kaba kuvvet, bütün çiftlere tek tek bakmak, aralarındaki uzaklığı hesaplamak ve en küçüğünü bulmaktır. Her eleman diğer $n-1$ elemanla karşılaştırılır; karmaşıklık n^2 mertebesindedir.

Daha iyi çözüm: noktalar önce $O(n \log n)$'de sıralanır. Sıralı dizide en yakın çift mutlaka ardışık iki noktadır; bu yüzden yalnız ardışık çiftlere bakmak yeterlidir, bu da $O(n)$'dir. Hoca iki adımın birbirine nasıl bağlı olduğunu sordu: çarpım şeklinde değil, toplam şeklinde, çünkü önce sıralanır, sonra taranır. Big-O gösteriminde toplamda büyük olan terim kalır:

$$O(n \log n) + O(n) = O(n \log n)$$

10.2 Bağlı Listede İkili Arama

İkili arama (binary search), sabit bir çarpanla bölme (divide by a constant factor) örneğidir. Sıralı bir dizide önce ortadaki eleman bulunur; aranan eleman ortadakinden büyükse sağ, küçükse sol yarı incelenir ve bu, eleman bulunana kadar sürer. Her seferinde dizi ikiye bölündüğü için karmaşıklık $\Theta(\log n)$'dir (ekrandan, 01:41:55).

Soru: Elemanlar yine sıralı tutulsa ama dizi yerine bağlı listede (linked list) tutulsaydı karmaşıklık ne olurdu?

Hocanın cevabı: Dizide ortadaki elemana her seferinde $O(1)$'de ulaşılır ve bu $\log n$ kez yapılır. Bağlı listede ortadaki elemanı bulmak bu kadar kolay değildir; listeyi baştan takip etmek gerekir ve bu $O(n)$'dir. Dizide $\log n$ adımın her biri sabit bir çarpanla çarpılırken bağlı listede n ile çarpılır; hoca sonucu $\log n \cdot n$, yani $n \log n$ olarak verdi.

> **Ders sonrası not:** $n \log n$ doğru bir üst sınırdır ama sıkı değildir. Her adımda ortaya yürümek yalnız kalan aralığın yarısı kadar sürer: $n/2 + n/4 + \cdots < n$. Toplam yürüme $O(n)$, karşılaştırma sayısı $O(\log n)$'dir; yani sıralı bağlı listede ikili arama $O(n)$'de biter ve doğrusal aramadan asimptotik olarak daha iyi değildir.

11. Tekrar: Ana Teorem

Hoca ana teoremin (master theorem) önemli olduğunu, biraz formül uygulamaya benzediğini söyleyerek kısaca hatırlattı (ekrandan, 01:44:55). Yineleme bağıntısı

$$T(n) = a\,T(n/b) + f(n)$$

biçimindeyse karmaşıklık; alt problem sayısı a, alt problemlerin küçülme oranı b ve alt problemleri birleştirmek için harcanan zamanı veren $f(n)$ ile ifade edilir. $f(n) \in \Theta(n^d)$ ise:

$$T(n) \in \begin{cases} \Theta(n^d) & a < b^d \\ \Theta(n^d \log n) & a = b^d \\ \Theta(n^{\log_b a}) & a > b^d \end{cases}$$

Hoca, bir yineleme bağıntısının çözümü için ana teoremin, özyineleme ağacının ve matematiksel tümevarımla yerine koymanın kullanılabildiğini, hangisinin daha uygun olduğuna karar vermek gerektiğini hatırlattı.

Örnek: $T(n) = 4T(n/2) + n$. Burada $a = 4$, $b = 2$, $d = 1$'dir. $b^d = 2$ ve $a = 4 > 2$ olduğundan üçüncü durum geçerlidir:

$$T(n) \in \Theta(n^{\log_2 4}) = \Theta(n^2)$$

12. Tekrar: Hash Fonksiyonları ve Çakışma

12.1 Beklenen Çakışma Sayısı

Hoca, hash fonksiyonlarında çakışma (collision) üzerine çok çalışıldığını ve hep benzer bir yaklaşımla hareket edildiğini hatırlattı. Anahtar sayısı n, boyutu m olan bir hash tablosuna anahtarlar düzgün hash'leme (uniform hashing) ile yerleştirilsin. Düzgün hash'lemede herhangi bir anahtarın diğerleriyle çakışma olasılığı aynıdır.

Beklenen çakışma sayısı (expected number of collisions) şöyle bulunur (ekrandan, 01:50:00): İki ayrı k ve l anahtarının çakışma olasılığı, tablo boyutu m olduğu için $1/m$'dir. Kaç farklı (k, l) çifti varsa bu sayı $1/m$ ile çarpılır. n anahtardan $\binom{n}{2}$ farklı ikili seçilebilir:

$$E[\text{çakışma}] = \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2} \cdot \frac{1}{m}$$

12.2 Uzun Dizgiler Tutan Bağlı Listede Arama

Hoca öğrencilerin üzerinde düşünmesini istediği bir problem verdi (ekrandan, 01:50:00): Bir bağlı listenin her düğümünde çok uzun dizgiler (long string) var ve elimizdeki bir anahtarın bu dizgilerden hangisine denk geldiği aranıyor. Bağlı listede arama yapıldığı için anahtar önce birinci dizgiyle karşılaştırılır, uymazsa bir sonraki düğüme geçilir, ve bu böyle sürer. Listede n eleman varsa maliyet $n \cdot$ dizgi uzunluğu kadardır.

Soru: Kullanılan yapılardan hangisi bu probleme eklenirse, listede ek olarak hangi bilgi tutulursa, arama dizgilerin bütün karakterleri karşılaştırılmadan yalnız liste üzerinde gezilerek yapılabilir?

Hocanın cevabı: Her düğümde dizgiyle birlikte o dizginin hash değeri de tutulur. Aranan anahtarın da hash değeri hesaplanır ve dizgiler yerine yalnız hash değerleri karşılaştırılır. Hoca hash hesaplamanın ve karşılaştırmanın $O(1)$'de yapıldığını kabul ederek işin yalnız bağlı listeyi gezmekle biteceğini söyledi.

Ek kontrol de gerekir: çakışma olabilir ve çakışma durumunda yanlış eleman döndürülebilir. Bu yüzden aranan anahtarın hash'i ile bir düğümde kayıtlı hash eşleşirse, çakışmaya denk gelme ihtimalini ortadan kaldırmak için yalnız o eşleşen düğümde dizginin tamamı anahtarla bir kez karşılaştırılır. Hoca bunun sözde kodunun yazılabileceğini, ama temel mantığı anlattığı için yazmayacağını söyledi.

13. Tekrar: Matematiksel Tümevarımla Yerine Koyma

Hoca, matematiksel tümevarımla yerine koyma (substitution) yöntemini bir örnekle hatırlattı. Seçilen bağıntı ana teoreme uyan bir bağıntıdır, ama açıklama açısından örnek güzel olduğu için tümevarımla çözüldü. Yöntemde bir tahmin yapılır ve tahmin ispatlanmaya çalışılır.

İspatlanacak bağıntı (ekrandan, 01:55:10):

$$T(n) = 4T(n/4) + n$$

13.1 Birinci Tahmin: $T(n) = O(n)$

Tahmin $T(n) = O(n)$'dir; yani bir c için $T(n) \leq cn$ ispatlanmalıdır. Varsayım: n'den küçük her m için $T(m) \leq cm$ doğrudur. Bu kabulle $T(n) \leq cn$ ispatlanmaya çalışılır. Bağıntı her seferinde 4'e bölerek gittiği için $m = n/4$ seçilir:

$$T(n/4) \leq c \cdot \frac{n}{4}$$

Bu, bağıntıda yerine konur:

$$T(n) = 4T(n/4) + n \leq 4 \cdot c\frac{n}{4} + n = cn + n$$

İspatlanmak istenen $T(n) \leq cn$ idi, ama bulunan $T(n) \leq cn + n$'dir. Bunu sağlayan bir $T(n)$, pozitif n ve c için cn sınırını sağlamayabilir. Demek ki tahmin yanlıştır; bulunan şey aslında $T(n)$'nin cn'den büyük olduğudur. Hoca, böyle bir sonucun bir fikir verdiğini söyledi: tahmin ispatlanamadıysa daha büyük bir sınırla denemek gerekir.

13.2 İkinci Tahmin: $T(n) \leq n \log_2 n$

Yine n'den küçük her m için $T(m) \leq m \log_2 m$ kabul edilir ve $m = n/4$ seçilir (ekrandan, 01:57:50):

$$T(n/4) \leq \frac{n}{4} \log_2 \frac{n}{4}$$

Logaritma dönüşümü: $\log_2(n/4) = \log_2 n - \log_2 4 = \log_2 n - 2$. Bağıntıda yerine konunca:

$$T(n) \leq 4 \cdot \frac{n}{4} \log_2 \frac{n}{4} + n = n(\log_2 n - 2) + n = n \log_2 n - 2n + n = n \log_2 n - n$$

İspatlanmak istenen $T(n) \leq n \log_2 n$ idi; bulunan $T(n) \leq n \log_2 n - n$ bundan da küçüktür. Tümevarım tuttu: $T(n) = O(n \log n)$.

Hoca, tahmin n^2 yapılsaydı da ispatın tutacağını, çünkü Big-O tanımı gereği daha büyük bir sınırın da doğru olacağını söyledi. Asıl soru daha sıkı bir sınır denenip denenemeyeceğidir.

13.3 Üçüncü Tahmin: $T(n) \leq n \log_4 n$

Bu kez $T(m) \leq m \log_4 m$ kabul edilir ve yine $m = n/4$ seçilir (ekrandan, 02:00:00):

$$T(n/4) \leq \frac{n}{4} \log_4 \frac{n}{4}$$

$\log_4(n/4) = \log_4 n - 1$ olduğundan (ara adım eklendi):

$$T(n) \leq 4 \cdot \frac{n}{4} \log_4 \frac{n}{4} + n = n \log_4 n - n + n = n \log_4 n$$

İspatlanmaya çalışılan $T(n) \leq n \log_4 n$ idi ve tamamen aynısı bulundu. Hoca bunu sıkı sınırı (exact bound) bulmak olarak yorumladı.

> **Ek bilgi (derste anlatılmadı):** Aynı bağıntıya ana teorem de uygulanabilir: $a = 4$, $b = 4$, $d = 1$ için $a = b^d$ olduğundan $T(n) \in \Theta(n \log n)$'dir. Bu, tümevarımla bulunan $n \log_4 n$ sınırıyla tutarlıdır; $\log_2 n$ ile $\log_4 n$ yalnız sabit bir çarpanla ($\log_2 n = 2 \log_4 n$) ayrıldığı için ikisi de aynı asimptotik sınıftadır.

14. Tekrar: Özyineleme Ağacı Yöntemi

Hoca, özyineleme ağaçlarıyla (solving recurrences by recursion tree method) çok örnek çözülmediği için iki örnek daha çözdü. Bu yöntemde bağıntı bir ağaca yerleştirilerek çözülür. Önceki derslerde Hanoi kuleleri ve Fibonacci ağaca yerleştirilmişti; orada ağacın dengeli olmadığı durumlar olmuş, altın oran karşıya çıkmıştı.

Yöntem şöyledir: $T(n)$'den başlanır. Her fonksiyon çağrısının yaptığı iş ve onun çağırdığı fonksiyon çağrıları ağaçta alt düğümlere yerleştirilir. Sonra her seviyenin işi toplanır; bunun için ağacın derinliği önemlidir.

14.1 Birleştirmeli Sıralama

Birleştirmeli sıralama (merge sort) bağıntısı (ekrandan, 02:03:40):

$$T(n) = 2T(n/2) + n, \quad T(1) = 0$$

$2T(n/2)$, iki yarıyı sıralamak için gereken işi; n, bölme ve birleştirme için gereken işi gösterir. $T(1) = 0$'dır, çünkü tek elemanda artık çağrı yapılmaz.

- Kök: $T(n)$, iki tane $T(n/2)$ çağırır ve kendisi n işlem yapar.
- 1. seviye: iki tane $T(n/2)$ vardır. Her biri iki tane $T(n/4)$ çağırır ve $n/2$ işlem yapar. Seviyenin toplam işi $2 \cdot n/2 = n$'dir.
- 2. seviye: dört tane $T(n/4)$ vardır; her biri $n/4$ işlem yapar. Toplam $4 \cdot n/4 = n$'dir.
- Bu böyle $T(1)$'lere kadar gider.

Her seviyede çarpan ile bölen birbirini götürür ve her seviyede n işlem vardır. Ağacın seviye sayısı $1 + \log_2 n$'dir. Örneğin $n = 8$ olsaydı boyutlar 8, 4, 2, 1 diye gider, ağaçta dört seviye olurdu. Yapraklar ($T(1) = 0$) iş yapmadığı için $\log_2 n$ seviyenin her biri n iş yapar ve toplam $n \log_2 n$ olur. Hoca sonucu $n \log n$ olarak verdi.

14.2 $T(n) = 2T(n/2) + n^2$

İkinci örnek (ekrandan, 02:06:55):

$$T(n) = 2T(n/2) + n^2$$

- 0. seviye: $T(n)$ kendisi n^2 işlem yapar.
- 1. seviye: iki tane $T(n/2)$ vardır, her biri $(n/2)^2$ işlem yapar: $2(n/2)^2 = n^2/2$.
- 2. seviye: dört tane $T(n/4)$ vardır, her biri $(n/4)^2$ işlem yapar: $4(n/4)^2 = n^2/4$.
- i'inci seviye: $n^2/2^i$.

Ağaç ikiye bölerek gittiği için derinliği $\log_2 n$'dir. Toplam:

$$T(n) = \sum_{i=0}^{\log_2 n} \frac{n^2}{2^i} = n^2 \sum_{i=0}^{\log_2 n} \frac{1}{2^i}$$

Seri sabit bir sayıyla sınırlıdır, bu yüzden $T(n) \in O(n^2)$'dir. Hoca, bu örnekte ağacın derinliğinin aslında çok önemli olmadığını, çünkü seviyelerin işi ikiye bölünerek çok hızlı azaldığı için derinliğin bir noktadan sonra önemsiz hâle geldiğini söyledi.

> **Ders sonrası not:** Hoca toplamı "birden küçük" diye yazdı (ekrandan, 02:06:55). $i = 0$ terimi zaten 1 olduğu için doğrusu $\sum_{i=0}^{\log_2 n} 1/2^i < 2$'dir. Sonuç değişmez: $T(n) < 2n^2$, yani $O(n^2)$. Kök tek başına n^2 iş yaptığı için sınır sıkıdır, $\Theta(n^2)$.

Örnek Sorular

Soru 1 (ekrandan, 00:50:00): Dinamik tabloda tablo dolunca boyut iki katına çıkarılıyor, doluluk $1/2$'nin altına inince yarıya indiriliyor. n 2'nin kuvveti olmak üzere, n işlemlik hangi ekleme-silme dizisi bu stratejide en kötü durumu verir ve toplam maliyet nedir?

> **Çözüm:** İlk $n/2$ işlem ekleme olsun ($\Theta(n)$; sonunda tablo $n/2$ boyutunda ve dolu). Sonraki $n/2$ işlem "ekle, sil, sil, ekle, ekle, sil, sil, ..." diye gelsin. Her ekleme çifti bir genişletme, her silme çifti bir küçültme yapar; her biri $\Theta(n)$ eleman taşır ve bunlardan $\Theta(n)$ tane olur. Toplam $\Theta(n^2)$'dir. Çözüm, küçültme sınırını $1/4$'e indirmektir.

Soru 2 (derste sözlü, 01:19:33): Düzeltilmiş stratejide (dolunca iki kat, $\alpha < 1/4$ olunca yarıya) $\Phi(T) = 2 \cdot T.num - T.size$ ($\alpha \geq 1/2$), $\Phi(T) = T.size/2 - T.num$ ($\alpha < 1/2$) ile, i'inci işlem silme ve $\alpha_{i-1} \geq 1/2$ iken amortize maliyeti bulun.

Çözüm (derste verilmedi): Silmede $num_i = num_{i-1} - 1$'dir. $\alpha_{i-1} \geq 1/2$ iken silme küçültmeye yol açmaz, çünkü bir eleman çıkınca doluluk $1/4$'ün altına inmez (tablo en az 4 yuvalı iken), bu yüzden $size_i = size_{i-1}$ ve $c_i = 1$'dir.

- $\alpha_i \geq 1/2$ ise: $\hat{c}_i = 1 + (2num_i - size_i) - (2(num_i + 1) - size_i) = 1 - 2 = -1$.
- $\alpha_i < 1/2$ ise (önceki doluluk tam $1/2$, yani $num_{i-1} = size_i/2$): $\hat{c}_i = 1 + (size_i/2 - num_i) - (2num_{i-1} - size_i) = 1 + 1 - 0 = 2$.

İki durumda da amortize maliyet sabittir (≤ 2). Böylece bütün ekleme ve silme durumları $O(1)$, n işlem $O(n)$'dir. Rastgele ekleme-silme dizileriyle yapılan python denemesinde bütün durumlarda gözlenen en büyük amortize maliyet 3 oldu ve potansiyel hiç negatife düşmedi.

Soru 3 (ekrandan, 01:40:00): Gerçek doğru üzerinde $x_1, \ldots, x_n$ noktaları var. Birbirine en yakın iki nokta kaba kuvvetle bulunursa karmaşıklık nedir? Daha iyi bir algoritma nasıl tasarlanır?

Çözüm: Kaba kuvvet bütün çiftlerin uzaklığına bakar: $O(n^2)$. Daha iyisi: önce sırala ($O(n \log n)$), sonra yalnız ardışık çiftlere bak ($O(n)$). Toplam $O(n \log n) + O(n) = O(n \log n)$.

Soru 4 (ekrandan, 01:41:55): İkili arama sıralı dizide $\Theta(\log n)$'dir. Elemanlar sıralı olarak bağlı listede tutulsaydı karmaşıklık ne olurdu?

Çözüm: Bağlı listede ortadaki elemana erişim $O(1)$ değil $O(n)$'dir, çünkü liste baştan yürünür. Hoca bunu $\log n$ adımın her birinin n ile çarpılması olarak yorumlayıp $O(n \log n)$ dedi. Yürüme mesafeleri her adımda yarıya indiği için ($n/2 + n/4 + \cdots < n$) daha sıkı sınır $O(n)$'dir (bölüm 10.2'deki ders sonrası not).

Soru 5 (ekrandan, 01:44:55): $T(n) = 4T(n/2) + n$ bağıntısını ana teoremle çözün.

Çözüm: $a = 4$, $b = 2$, $d = 1$; $b^d = 2 < a$. Sonuç $\Theta(n^{\log_2 4}) = \Theta(n^2)$.

Soru 6 (ekrandan, 01:50:00): n anahtar, m boyutlu bir hash tablosuna düzgün hash'leme ile yerleştiriliyor. Beklenen çakışma sayısı nedir?

Çözüm: Her farklı çift $1/m$ olasılıkla çakışır ve $\binom{n}{2}$ çift vardır: $\frac{n(n-1)}{2} \cdot \frac{1}{m}$.

Soru 7 (ekrandan, 01:50:00): Her düğümünde çok uzun bir dizgi olan bir bağlı listede bir anahtar aranıyor; maliyet $n \cdot$ dizgi uzunluğu. Listede ek olarak hangi bilgi tutulursa arama dizgilerin bütün karakterleri karşılaştırılmadan yapılabilir? Hangi ek kontrol gerekir?

Çözüm: Her düğümde dizginin hash değeri de tutulur. Anahtarın hash'i bir kez hesaplanır ve gezilen her düğümde yalnız hash değerleri karşılaştırılır. Hash'ler eşleşince, çakışma sonucu yanlış elemanın döndürülmemesi için yalnız o düğümde dizgi anahtarla bir kez tam olarak karşılaştırılır.

Soru 8 (ekrandan, 01:55:10): $T(n) = 4T(n/4) + n$ için matematiksel tümevarımla sırasıyla $T(n) \leq cn$, $T(n) \leq n \log_2 n$ ve $T(n) \leq n \log_4 n$ tahminlerini sınayın.

Çözüm: $m = n/4$ için varsayım yerine konur. cn: $T(n) \leq cn + n$, tahmin tutmaz. $n \log_2 n$: $T(n) \leq n(\log_2 n - 2) + n = n \log_2 n - n$, tutar. $n \log_4 n$: $T(n) \leq n(\log_4 n - 1) + n = n \log_4 n$, tutar ve ispatlanmak istenenin aynısı çıkar.

Soru 9 (ekrandan, 02:03:40): Birleştirmeli sıralamanın $T(n) = 2T(n/2) + n$, $T(1) = 0$ bağıntısını özyineleme ağacıyla çözün.

Çözüm: Her seviyenin toplam işi n'dir (n, $2 \cdot n/2$, $4 \cdot n/4$, ...). Ağaçta $1 + \log_2 n$ seviye vardır; yapraklar ($T(1) = 0$) iş yapmaz. Toplam $n \log_2 n$, yani $\Theta(n \log n)$.

Soru 10 (ekrandan, 02:06:55): $T(n) = 2T(n/2) + n^2$ bağıntısını özyineleme ağacıyla çözün.

Çözüm: i'inci seviyenin işi $2^i(n/2^i)^2 = n^2/2^i$'dir. Toplam $n^2 \sum_{i=0}^{\log_2 n} 1/2^i < 2n^2$. Sonuç $O(n^2)$; kök tek başına n^2 iş yaptığı için $\Theta(n^2)$.

Dersin Toparlaması

Dersin ilk yarısında, önceki derste öğrenilen üç amortize analiz yöntemi dinamik tabloya uygulandı. Yalnız ekleme varken tablo dolunca boyutu iki katına çıkarmak, toplam analizle n ekleme için $3n$'den küçük bir toplam maliyet verdi. Muhasebe yönteminde her eklemeye 3 kredi biçildi; potansiyel yönteminde $\Phi = 2 \cdot num - size$ ile her eklemenin amortize maliyeti 3 çıktı. Silme işin içine girince, küçültme sınırını $1/2$ seçmenin ekleme ve silmeler arasında gidip gelen bir diziyle $\Theta(n^2)$ maliyete yol açtığı görüldü. Küçültme sınırı $1/4$'e indirilip potansiyel fonksiyonu iki parçalı tanımlanınca ekleme ve silmelerin her durumunda amortize maliyet sabit kaldı; n işlem $O(n)$ tutar. Analizin kilit noktası, her durumda işlemin genişletmeye ya da küçültmeye yol açıp açmadığını belirleyip eşitlikleri buna göre kurmaktır.

İkinci yarı sınav öncesi bir tekrardı: en yakın çift problemi ve bağlı listede ikili arama üzerinden karmaşıklıkların toplanması ve çarpılması, ana teoremin üç durumu, düzgün hash'lemede beklenen çakışma sayısı, uzun dizgiler tutan bir bağlı listede aramanın hash değerleriyle hızlandırılması, tümevarımla yerine koymada tahminin adım adım sıkılaştırılması ve özyineleme ağacıyla iki bağıntının çözümü.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.